Máster en Desarrollo con Inteligencia Artificial · Big School

Trabajo Fin de Máster · Accesibilidad 360

Anexo D. Modelo de datos

Autor: Jorge Serna

Tutor: Brais Moure

Anexo D. Modelo de datos

Este anexo recoge el modelo de datos de la aplicación tal como está definido en el esquema de Prisma, junto con el diagrama entidad-relación de las nueve entidades de dominio y la relación de migraciones. El diseño y sus decisiones se justifican en el capítulo 5 de la memoria.

D.1 Diagrama entidad-relación

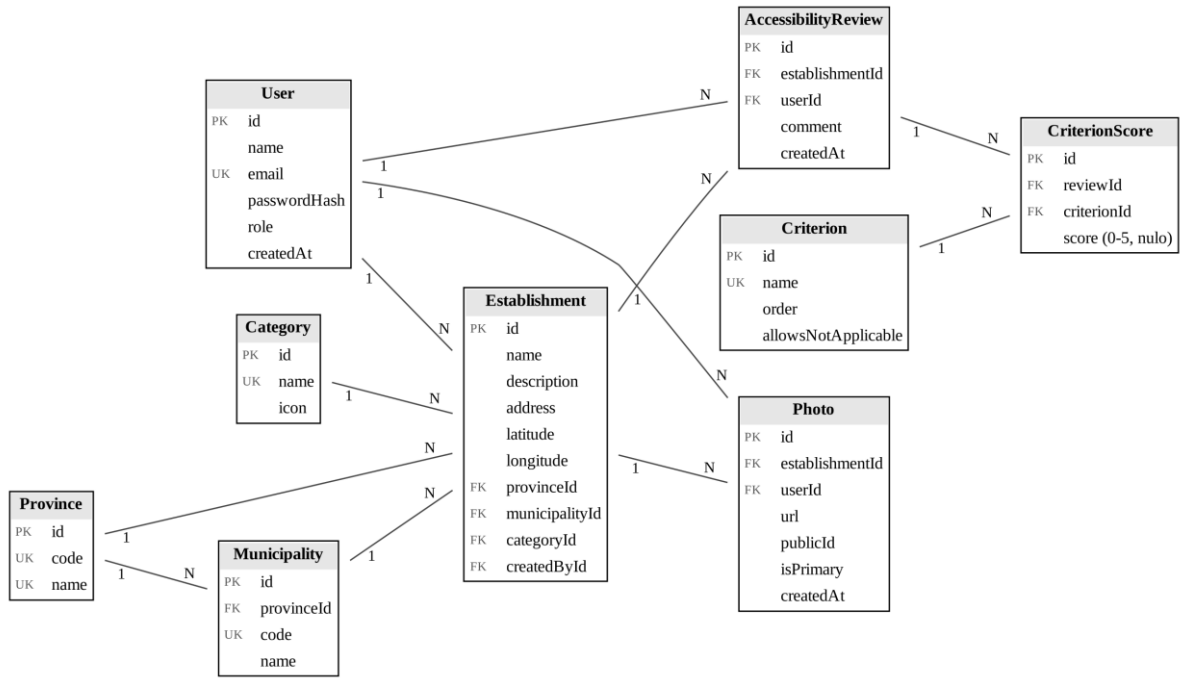


Figura D.1. Modelo entidad-relación de las entidades de dominio.

Las tres tablas que Auth.js define en el esquema (cuentas, sesiones y tokens de verificación) no se muestran porque la aplicación no las utiliza (capítulo 5).

D.2 Migraciones

El esquema se ha construido con 8 migraciones, que se aplican en orden con `prisma migrate deploy`:

1. `20261007100144_init_auth`
2. `20261007143729_add_establishments`
3. `20261007153948_add_accessibility_reviews`
4. `20261008075329_add_photos`
5. `20261008111623_add_establishment_coordinates`
6. `20261008113426_add_geo_entities`

7. `20261008113634_drop_geo_text_columns`
8. `20261008151738_add_criterion_not_applicable`

D.3 Esquema de Prisma

Contenido íntegro del archivo `prisma/schema.prisma`:

```
// Esquema Prisma (SPEC-010 fase 010.2: modelo de datos de autenticación).
// La primera migración se generará cuando exista la base de datos Neon,
// durante la fase de despliegue.

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE URL")
}

// Roles del dominio (001-domain-model.md + SPEC-010).
// Todo usuario nuevo recibe USER por defecto.
enum Role {
  USER
  MODERATOR
  ADMIN
}

// Usuario registrado.
//
// - `name` es obligatorio en el MVP porque el registro mediante
//   Credentials siempre solicita el nombre del usuario.
// - `passwordHash` es nullable para permitir una futura incorporación
//   de proveedores OAuth sin requerir una migración del esquema.
// - Nunca se almacenarán contraseñas en texto plano.
model User {
  id            String    @id @default(cuid())
  name          String
  email         String    @unique
  emailVerified DateTime?
  image         String?
  passwordHash  String?
  role          Role      @default(USER)
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  accounts Account[]
  sessions Session[]

  establishments Establishment[]
  reviews        AccessibilityReview[]
  photos         Photo[]
}

// Requerido por Auth.js v5. Sin campos adicionales.
model Account {
  id                String  @id @default(cuid())
  userId            String
  type              String
  provider          String
  providerAccountId String
  refresh_token     String? @db.Text
  access_token      String? @db.Text
  expires_at        Int?
  token_type        String?
  scope             String?
  id_token          String? @db.Text
  session_state     String?

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
```

```
  @@unique([provider, providerAccountId])
}

// Requerido por Auth.js v5. Sin campos adicionales.
// Se utilizará cuando se activen las sesiones de base de datos;
// en esta fase las sesiones son JWT (ver lib/auth/auth.config.ts).
model Session {
  id           String   @id @default(cuid())
  sessionToken String   @unique
  userId       String
  expires      DateTime

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}

// Categoría de establecimiento (SPEC-030).
// `name` es único para permitir un seed idempotente por nombre.
// `icon` es opcional (nombre del icono Lucide, sin administrar todavía).
model Category {
  id        String   @id @default(cuid())
  name      String   @unique
  icon      String?
  createdAt DateTime @default(now())

  establishments Establishment[]
}

// Establecimiento físico, primera entrega (SPEC-030).
// Solo datos básicos de ficha: sin fotografías, valoraciones,
// puntuación, coordenadas ni accesibilidad (fases posteriores).
// `postalCode` existe como opcional pero no se solicita en el formulario.
model Establishment {
  id             String   @id @default(cuid())
  name           String
  description    String?
  address        String
  postalCode     String?
  latitude       Float?
  longitude      Float?
  provinceId     String
  municipalityId String
  categoryId     String
  createdById    String
  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt

  category     Category     @relation(fields: [categoryId], references: [id], onDelete:
Restrict)
  createdBy    User         @relation(fields: [createdById], references: [id], onDelete:
Cascade)
  province     Province     @relation(fields: [provinceId], references: [id], onDelete:
Restrict)
  municipality Municipality @relation(fields: [municipalityId], references: [id], onDelete:
Restrict)

  reviews AccessibilityReview[]
  photos  Photo[]
}

// Provincia española (SPEC-035, Fase 1).
// `code`: código oficial INE de 2 dígitos. `name`: denominación oficial.
// La columna de texto `Establishment.province` coexiste hasta la Fase 3.
model Province {
  id   String @id @default(cuid())
  code String @unique
  name String @unique

  municipalities Municipality[]
  establishments Establishment[]
}

// Municipio español (SPEC-035, Fase 1).
// `code`: código oficial INE de 5 dígitos (provincia + municipio).
// Unicidad compuesta: hay nombres repetidos entre provincias.
// La columna de texto `Establishment.municipality` coexiste hasta la Fase 3.
model Municipality {
  id         String @id @default(cuid())
```

```
  provinceId String
  code       String @unique
  name       String

  province       Province        @relation(fields: [provinceId], references: [id], onDelete:
Restrict)
  establishments Establishment[]

  @@unique([provinceId, name])
}

// Criterio de accesibilidad (SPEC-040 + SPEC-045).
// Versión simplificada del dominio: sin code, weight, active ni icon.
// `name` es único para permitir un seed idempotente.
// `allowsNotApplicable`: solo algunos criterios admiten «No aplicable»
// (se excluyen de las medias); `null` en la puntuación lo representa,
// nunca un valor mágico.
// `order` define el orden de presentación en el formulario.
model Criterion {
  id                  String   @id @default(cuid())
  name                String   @unique
  description         String?
  order               Int
  allowsNotApplicable Boolean  @default(false)
  createdAt           DateTime @default(now())

  scores CriterionScore[]
}

// Valoración de un establecimiento por un usuario (SPEC-040).
// Un usuario solo puede tener una valoración por establecimiento
// (@@unique). El comentario es opcional; las puntuaciones por
// criterio viven en CriterionScore.
model AccessibilityReview {
  id              String   @id @default(cuid())
  establishmentId String
  userId          String
  comment         String?
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  establishment Establishment @relation(fields: [establishmentId], references: [id], onDelete:
Cascade)
  user          User          @relation(fields: [userId], references: [id], onDelete: Cascade)

  scores CriterionScore[]

  @@unique([establishmentId, userId])
}

// Puntuación de un criterio dentro de una valoración (SPEC-040 + SPEC-045).
// Entero de 0 a 5 (rango validado en Zod y servicio).
// `null` significa «No aplicable» (solo en criterios permitidos).
model CriterionScore {
  id          String @id @default(cuid())
  reviewId    String
  criterionId String
  score       Int?

  review    AccessibilityReview @relation(fields: [reviewId], references: [id], onDelete:
Cascade)
  criterion Criterion           @relation(fields: [criterionId], references: [id], onDelete:
Restrict)

  @@unique([reviewId, criterionId])
}

// Fotografía de un establecimiento (SPEC-050, 1.ª entrega).
// Solo se persisten `url` (secure url) y `publicId`; el archivo vive
// en Cloudinary (ADR-004). `isPrimary` prepara el soporte futuro de
// múltiples fotografías. La regla "una por establecimiento" vive en
// el servicio, no en base de datos.
model Photo {
  id              String   @id @default(cuid())
  establishmentId String
  userId          String
  url             String
  publicId        String
```

```
  caption         String?
  isPrimary       Boolean  @default(true)
  createdAt       DateTime @default(now())

  establishment Establishment @relation(fields: [establishmentId], references: [id], onDelete:
Cascade)
  user          User          @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([establishmentId])
}

// Requerido por Auth.js v5. Sin campos adicionales.
model VerificationToken {
  identifier String
  token      String   @unique
  expires    DateTime

  @@unique([identifier, token])
}
```